

Funkcionalno programiranje

Momir Adžemović, Gavriilo Jovanović

1. Uvod

Funkcionalno programiranje je paradigma čije su karakteristike:

- **Funkcije višeg reda** - Funkcije čiji je jedan od argumenata funkcija ili je povratna vrednost funkcija.
- **Imutabilnost** - Osobina objekta da ne može da se menja stanje nakon što je kreiran.
- **Čista funkcija(pure)** - Funkcija koja za isti upaz uvek daje isti izlaz i koji nema sporednih efekata.
- **Lenjost** - Izračunavanje je odloženo dok ne postane neophodno.
- **Stroga tipiziranost** - Promenljive imaju svoj tip kojim su ograničene. Prilikom deklaracije promenljive, dodeljujemo joj tip. Npr. u haskell-u dodeljujemo imena izrazima. Kada dodelimo ime izrazu A, ne možemo opet to ime dodeliti i izrazu B.

Jezici koji podržavaju funkcionalne jezike su:

- *Lisp* - prvi funkcionalni jezik.
- **Haskell** - današnji sinonim za FP.
- *Scala* - mešavina OO i FP.
- *Erlang* - jezik za distribuirane sisteme.
- *Elm* - jezik za pisanje veb aplikacija.
- **C++** - najpopularniji jezik sa funkcionalnim karakteristikama.

1.1. Uvod u napredniji haskel

Repna rekurzija je rekurzija gde se rekurzivni poziv izvršava na kraj. Repnu rekurziju možemo pretvoriti u petlju, što je lakše optimizovati.

Primer 1: Napisati haskell kod koji računa sumu elemenata liste preko repne rekurzije

```
sum xs =
    let helper acc values =
        case values of
            [] -> acc
            (y:ys) -> helper (acc+y) ys
    in helper 0 xs
```

Ovaj kod radi na sledeći način:

```
sum [1,2,3] = helper 0 [1,2,3] = helper (0+1) [2,3] = helper (1+2) [3] =
helper (3+3) [] = (6 + 0) = 6
```

Kao što vidimo, u haskell-u se koriste **lambda funkcije**, koje su anonimne funkcije i imaju sledeću sintaksu:

```
(\x y -> x + y) -- anonimna funkcija sabiranja
                -- koja uzima argumente i daje rezultat
```

Primer 2: Napisati haskell kod koji računa proizvod elemenata liste preko repne rekurzije

```
mul xs =
    let helper acc values =
        case values of
            [] -> acc
            (y:ys) -> helper (acc*y) ys
    in helper 1 xs
```

Kao što vidimo u *Primer 2* smo samo promenili inicijalnu vrednost akumulatora *acc* i operator.

Kao što smo isto videli u *Primer 1* naša operacija *sum* kreće od 0, pa 1, pa 3, pa 6. Takve operacije zovemo *leve asocijativne*. Da je *sum* funkcija radila tako da je *acc* prvo 3, pa 5, pa 6, onda bi ona bila *desno asocijativna*. Ovo možemo uopštiti na sledeći način.

Sve kolekcije u haskellu koje su tipa liste se nazivaju **foldable**. Sada ćemo da vidimo neke *nerekurzivne* funkcije koje rade nad ovim tipom, **foldr** i **foldl**. Ove **fold** tip funkcija "sabija" kolekciju u rezultat. Ekvivalent ovoj funkciji u C++ je *accumulate* funkcija.

```
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
foldl :: Foldable t => (b -> a -> b) -> a -> t a -> b (*)
```

Ove dve funkcije se razlikuju po smeru "sabijanja", tj. *asocijativnosti*. **foldl** je levo asocijativan dok je **foldr** desno asocijativan. Hajmo sada da vidimo detaljno elemente **foldl-a**(*):

- $(b \rightarrow a \rightarrow b)$ - Operator koji uzima akumulator tipa b i element tipa a kao argumente, i vraća rezultat tipa b .

Primer 3: $(\backslash \backslash \text{acc } x \rightarrow \text{acc} + x)$ ili $(+)$ koji već postoji u haskell-u. Za foldr je jedina razlika u redosledu primenjivanja operacije, tj. $(\backslash x \text{ acc} \rightarrow x + \text{acc})$.

- a - Inicijalna vrednost akumulatora.
- $t a$ - Kolekcija koja nasledjuje **Foldable** tj- može da se primenjuje na nju(Foldable t).
- *Drugo* b - Rezultat.

Prodjimo sada preko nekih osnovnih funkcija:

- $(\$)$: Odredjuje grupisanje naše funkcije. U haskell-u je uzimanje operatora levo asocijativno i ovako izgleda korišćenje $\$$ operatora:

$$1. f \ g \ x \iff (f \ g) \ x \qquad 2. f \ \$ \ g \ x \iff f \ (g \ x)$$

- $(.)$: Kompozicija funkcije: $f \ (g \ x) \iff f.g \ x$
- $(:)$: Desno asocijativna operacija koja dodaje element na početak liste:
- $(++)$: Operator konkatanacije liste niski

Primer 4: Kako preko *fold-a* dobiti od liste istu tu listu?

```
foldr (:) [] [1,2,3]
```

Objašnjenje: Operacija ":" je desno asocijativna. Foldr uzima desni element liste i dodaje ga na početak, tj. ceo proces biće: $(:) [] [1,2,3] \implies (:) [3] [1,2] \implies (:) [2,3] [1] \implies (:) [1,2,3] []$, u akumulatoru se nalazi naša rezultujuća lista.

1.1.1. Korisničke liste

Definicija: Rekurzivna lista je ona lista koja je ili prazna ili se sastoji iz glave(singularni element) i repa(ostali elementi).

Da bismo uveli korisničku definisanu listu, moramo da uvedemo ključnu reč koja definiše bilo koji tip. Takva reč je *data*. U haskell-u, tipovi se pišu velikim slovima.

```
data IList = Empty           -- prazna lista
           | Cons Int IList  -- glava i rep
           deriving Show     -- IList nasledjuje Show da bismo mogli
da prikazemo                -- objekat na izlazu kao nisku
```

Konstruktor tipa (IList): Uzima nula ili više argumenata.

Konstruktor vrednosti (`Empty`, `Cons`): Uzima nula ili više vrednosti (`Empty` - nula, `Cons` - glavu i rep koja predstavlja listu).

Primer 1: Univerzalna lista

```
data UList a = Empty
             | Cons a (UList a)
             deriving Show
```

Primer 2: Dimenzija liste

```
depth :: UList a -> Int
depth Empty = 0
depth (Cons a b) = 1 + depth b
```

Svojstvo listi, kao i ostalih objekata u haskell-u je **imutabilnost**. Kada imamo neku listu xs , i želimo da napravimo listu ys na sledeći način, $ys = y : xs$. Operator će samo da uzme referencu na nisku xs i da joj prilepi y , ne vrši se kopiranje. Zbog toga je memorijska i vremenska složenost $O(1)$. Ako bismo brisali sa početka liste, radili bismo u istoj složenosti.

Napomena: Kada bismo uradili operaciju konkatacije, tj. $xs ++ [y]$, vremenska i memorijska složenost bi bile $O(n)$, baš zbog imutabilnosti! Pogledajmo malo detaljnije na sledećem primeru.

Primer 3: Konkatanacija dve liste. Neka imamo xs i ys , gde važi da je $len(xs) = n$, i $len(ys) = m$. Trebamo da dobijemo $zs = xs ++ ys$. Kako ovo radi?

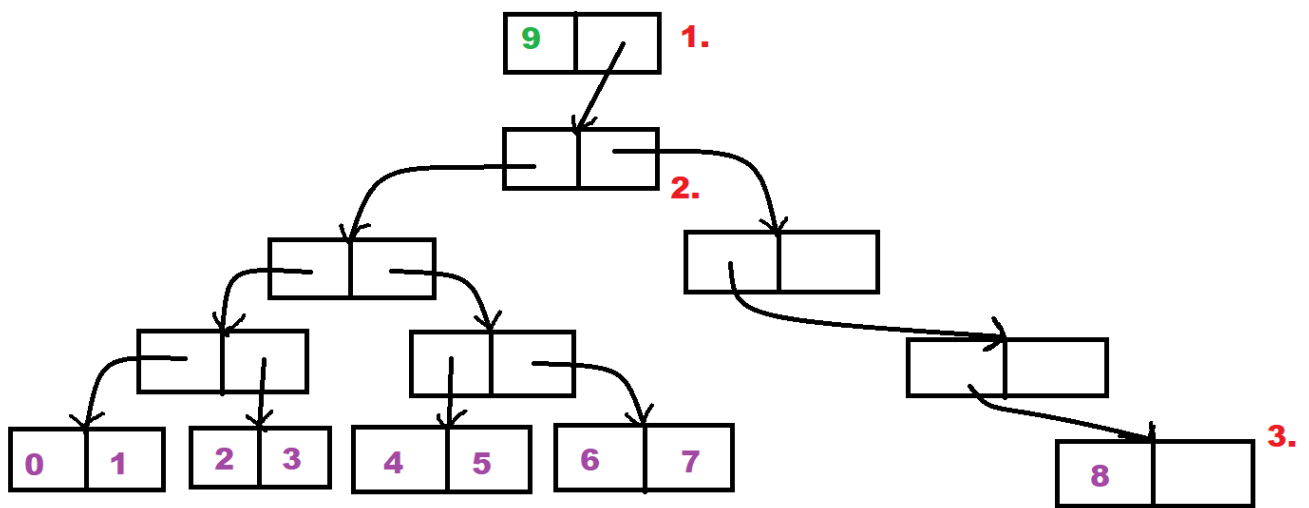
Prvo će da se kopira samo xs list, a rep ys ostaje zajednički za deo ys i zs . Pošto samo kopiramo xs , vremenska i memorijska složenost ove konkatanacije je $O(n)$. $\diamond$

Na žalost, liste imaju još mana. Kao i kod C-a, indeksiranje liste (operacija `!!`) se isto primenjuje u složenosti $O(n)$. Zbog rasporeda u memoriji, koji nije blokovski, liste imaju potencijalnog neiskorišćenog keša, koji nam usporava računar.

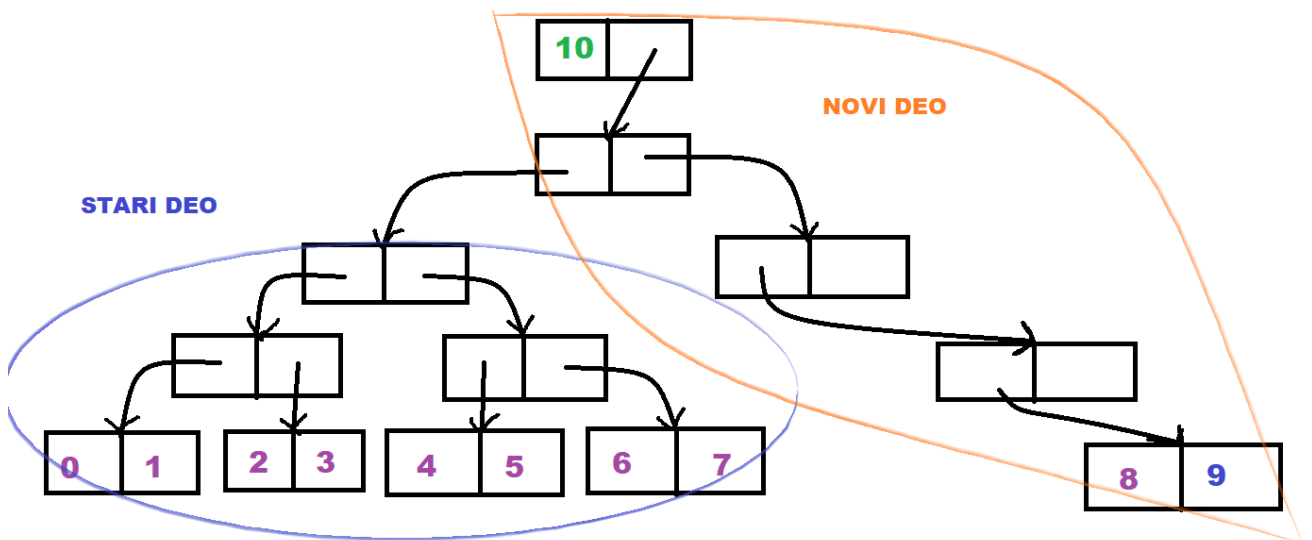
1.1.2. Persistent vector

Ovo dole je dat je ceo prikaz strukture vektora koji sadrži elemente `[0,1,2,3,4,5,6,7,8]`. Pojmovi označeni crvenim:

1. **Vektor** - Sadrži dimenziju strukture i pokazivač na koren strukture
2. *Koren* - Član koji sadrži pokazivače ka ostalim elementima, koji imaju pokazivače do listova
3. *List* - Sadrži jedan ili dva elementa.

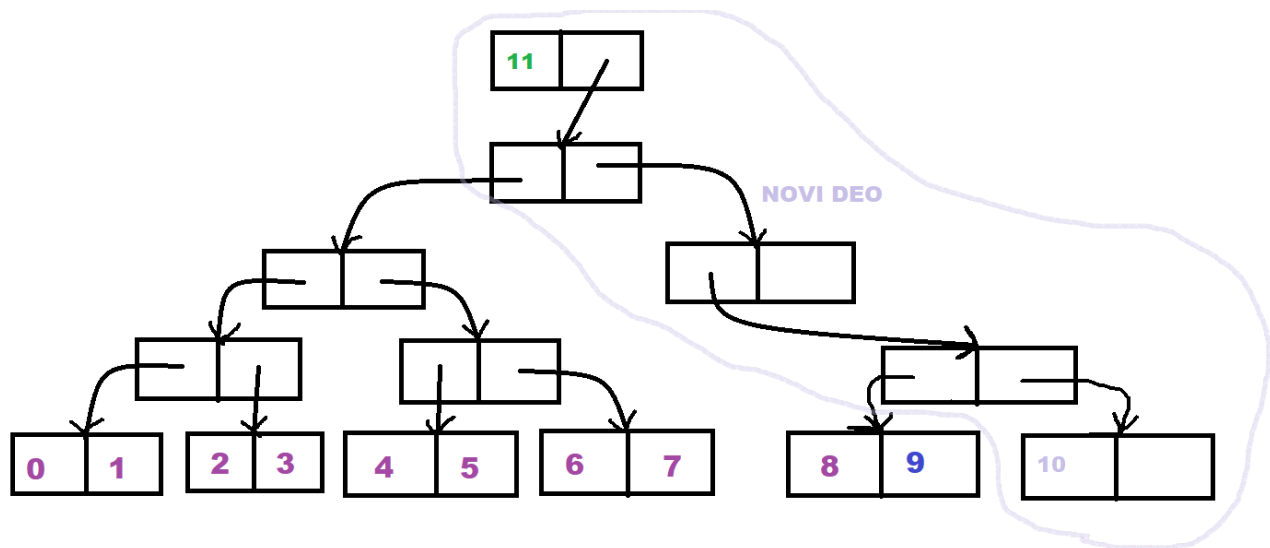


Kako dodajemo element u ovaj vektor? Zamislimo da trebamo da dodamo broj 9. Očigledno, u poslednjem listu pored 8, trebamo da je ubacimo. Kako ovo radimo? Rešenje je da kreiramo novi vektor gde čuvamo one stare delove vektora koje ne menjamo, a sve pretke od lista koji menjamo do korena da izmenimo. U našem slučaju to izgleda ovako:



Izmena elemenata je analogno dodavanju, tj. menjamo čvorove od početka to odgovarajućeg lista, tako da se ovde nećemo zadržati.

Šta se događa ako nema slobodnog mesta u listu, a dodajemo element? U visini stabla, pravimo novi list koji će da se nalazi u nivou listova i da sadrži element.



Šta se događa ako nema slobodnog mesta za kreiranje novog lista u poslednjem nivou stabla? Pa jednostavno, povećavamo njegov nivo onda.

Brisanje elemenata vekotra se svodi na inverzne slučajeve dodavanja novog elementa.

Složenost: Ako su čvorovi dimenzije 2, onda je složenost $O(\log_2 n)$. Ako fiksiramo dužinu stabla na visinu između 7-10, onda je složenost u teoriji $O(c)$, tj. $O(1)$. Ova struktura je sporija od klasičnog vektora do na konstantan faktor.

2. Sakupljači otpadaka

Iako smo navikli da operišemo memorijom, na nekim nivoima programiranje je suviše i može da dovodi do raznoraznih bagova, kao i nepogodnosti alokacije memorije na hipu. Zbog toga uvodimo **sakupljače otpadaka** (*garbage collector*). Stvari koje nam nisu potrebne moraju da se brišu iz memorije! Metoda sakupljača za nas radi posao automatski u haskell-u.

Sakupljači otpadaka mogu da rade na različite načine, ili da mešaju metode. Sada ćemo da prodjemo kroz njih.

I Brojanje referenci

Ova metoda radi tako što broji koliko stvari pokazuje na neki objekat pomoću brojača referenci. Ako na objekat ništa ne pokazuje, da li možemo da mu pristupimo? Ne, i zato želimo da ga obrišemo. Ako je brojač jednak nuli, onda brišemo objekat. Svako kopiranje objekta je samo povećavanje referenci na taj objekat.

Primer 1: Kako će da radi ova metoda na nekim objektima:

- Povezana lista - Ako brišemo objekat koji referiše na prvi član liste, onda se i on briše. Rekursivno se tako briše i cela lista.
- Drvo - slično kao i za listu
- Kružna lista - Nema pokazivač na strukturu, i broj referencijalnih ovih objekata nije nula pa se ne briše.
- Dvostruko povezana lista - Isti problem.

Prednosti koje imaju brojanje referenci su:

1. *Determinizam* - U trenutku kada niko ne koristi objekat, on biva uništen.
2. *Efikasnost* - Efikasne su operacije.
3. *Memorijska lokalnost* - Brojači se pamte zajedno sa objektom.

Mane koje imaju brojanje referenci su:

1. *Ciklične strukture* - Pogledati Primer 1.
2. *Mutabilne strukture i sve operacije su mutabilne* - Brojači se konstantno ažuriraju što narušava koncept imutabilnosti
3. *Fragmentacija memorije*
4. *Nemamo kontrolu* - Objekat se uništava u trenutku kada ništa ne pokazuje na njega. To može da dovede do lančanog uništavanja objekata, što može da uspori neki kritičan deo programa.

Primer 2: (Mana mutabilnosti) Recimo da imamo dva imutabilna objekata A i B tako da A ima referencu na B. U kom redosledu ih kreiramo?

Prvi slučaj: Prvo kreiramo A. Ovde je problem što ne možemo postaviti referencu na B, jer ne postoji trenutni.

Drugi slučaj: Prvo kreiramo B. Ovde je problem što ne možemo da postavimo referencu sa A, jer još nije kreiran.

U savršenom čistom jeziku nema ciklusa, i zato ne postoje čist funkcionalni jezik, jer oni postoje na neki način!

5. *Nije sakupljač podataka* - Pošto se objekat odmah briše čim ništa ne pokazuje na njega, onda taj objekat nije dubre u nekom trenutku.

II Optimizovan brojač referenci

Koristi prethodni koncept uz sledeće dodatke:

- *Odloženo ažuriranje brojača*
- *Spajanje uzastopnih promena brojača*

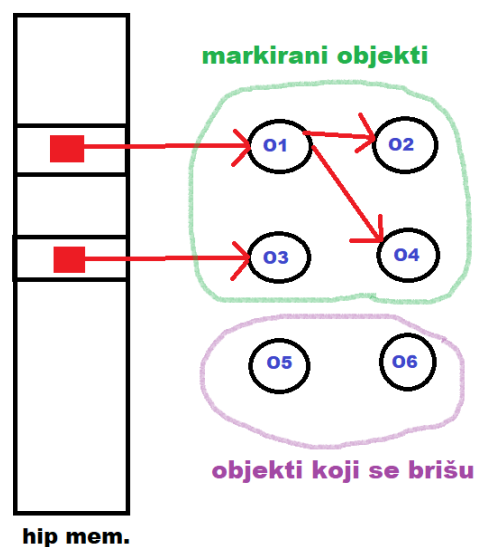
- *Baferovanje uzastopnih promena brojača*

Ako u nekom čvoru brojač referenci za objekat postane nula, ne uništavamo i taj objekat postaje džubre. Time brojač referenci postaje *garbage collector*.

III Mark-and-Sweep

Ovo je aparat koji određuje i briše džubre kroz dva koraka:

1. Sakupljamo sve pokazivače sa steka koji formiraju neki graf i obilazimo ga (npr. DFS) gde u svakom čvoru obeležavamo objekte na hipu, jer su inicijalno neobeleženi. Svi elementi koji ostanu neobeleženi su po definiciji džubre. Ovaj korak nazivamo **"mark"**.
2. Prolazimo kroz objekte na hipu i brišemo ih ako nisu markirani. Ovaj korak se naziva **"sweep"**. Svi objekti postaju neobeleženi nakon iove iteracije.



Ne možemo da znamo da li je nešto džubre na steku, ali možemo da znamo da li je džubre na hipu. Stek sam čisti svoje džubre. Pošto nemamo direktan pristup hipu, moramo da krenemo od steka.

Mark-and-sweep ima svojih prednost:

- *Rad sa ciklusima.*
- *Ne usporava program prilikom kopiranja* - Jeftino kopiranje.
- *Ne usporava program kada objekat postane džubre.*

Ali i svojih mana:

- **Stop the world** - U trenutku kada brišemo džubre, želimo da održimo konzistentnost. Ako brišemo džubre u realnom vremenu, možemo da izgubimo njihove pokazivače na druge objekat. Zato moramo da zamrznemo svet, i da obrišemo svo džubre pre nastavka programa. Brisanje se najčešće primenjuje kada ponestane slobodne memorije, što je pored svega, isto mana.
- *Lokalnost memorije i fragmentacija*

Izgubili smo determinizam, a dobili smo efikasnost. U svakom koraku imamo promenu u kešu. jer svaki put proveravamo objekat na drugoj memorijskoj poziciji.

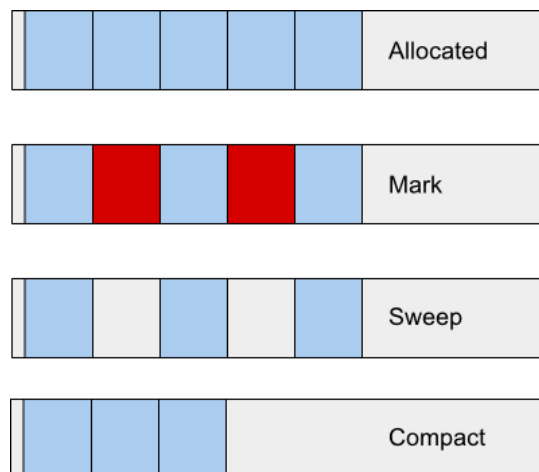
IV Mark-and-Compact

Ovo je modifikacija prethodnog algoritma, uz akcenat na rešavanje fragmentacije. Uz "sweep" deo koraka, vršimo i "**compact**". To je kada svu memoriju u hipu spojimo tako da ne bude "rupa".

Problem je što ovo nije fejtina operacija, ali nije sa druge strane ni Mark-and-Sweep.

Kako vršimo kompaktiranje? Radimo "*sliding*" objekata i time možemo a i ne moramo da čuvamo poredak. Kada nam poredak nije bitan, možemo da ubacujemo objekat u slobodan prostor izmedju dva prethodna objekta, što je efikasnije. Ako koristimo drugu metodu, moramo da pazimo, jer na kraju skoro sigurno ostaje particionisanje. Zato uvek kao poslednje operacije radimo *sliding* manevar. Posle ovoga imamo malu fragmentaciju i veoma je lokalizovana. Problem je samo što je ovo veoma skupa operacija za sakupljanje đubreta.

Jedna od tehnike kompresije je *two finger compaction* koja nam rešava potencijalni problem fragmentacija ako ne želimo da čuvamo poredak. Particionisanjem sa dva pokazivača(početak/kraj), ova metoda je pogodna za objekte iste veličine. Ovo je zato znatno sporiji algoritam.



Mark-and-compact

V Kopirajući sakupljač đubradi

Ovom metodom svaki put kopiramo sve. Razdvajamo memoriju na dva dela, *fromspace* i *tospace*. Kopiramo objekte sa *fromspace* u *tospace*. Na kraju obrćemo *fromspace* i *tospace*.

Obilazimo sve objekte kao u "mark" i umesto obeleživanja, prebacimo ih u drugi deo. Ne moramo da radimo bilo šta, dovoljno je da ignorišemo stari deo.

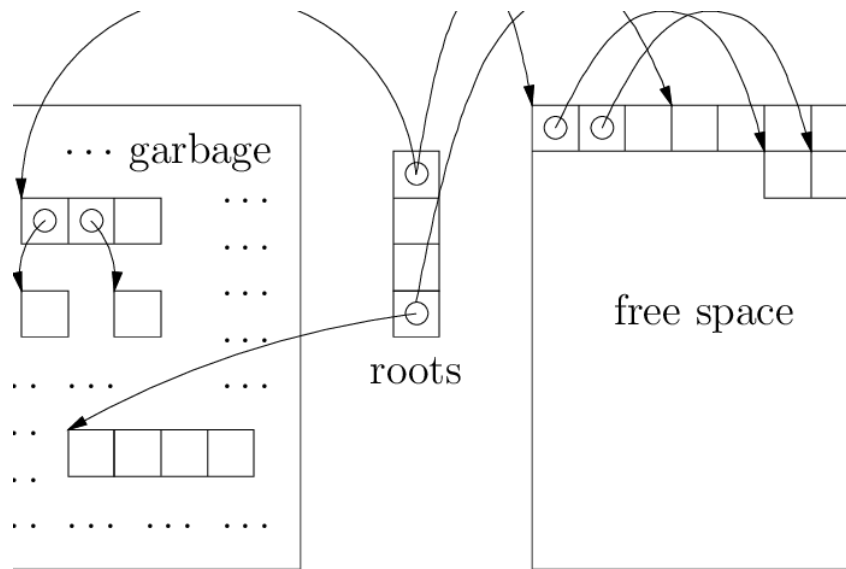
Smanjuje se "sweep" deo, jer ne moramo da obilazimo kroz sve objekte na hipu. Dosta efikasna tehnika za programe koje imaju mnogo đubreta, a malo podataka.

Prednosti:

- *Lokalnost*
- *Jednostavno sakupljanje objekata*
- *Brza alokacija*
- *Mala fragmentacija* - Prilikom kopiranja vršimo sabijanje

Mane:

- *Duplo manja memorija*
- *Kopiranje podataka*
- *Kopiranje velikih i trajnih objekata*



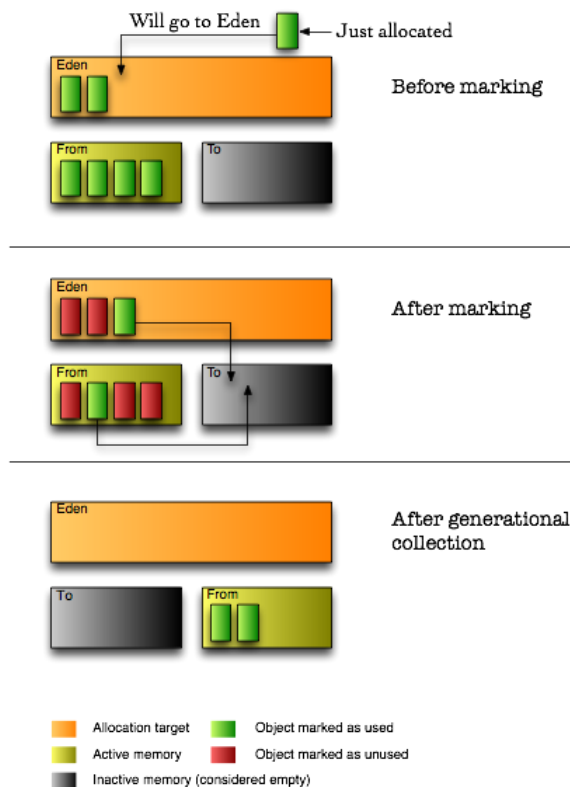
Kopirajući sakupljač đubradi

VI Generacijski sakupljač đubradi

Ovaj sakupljač radi na sledećim pretpostavkama:

- Najmlađi objekti, koji su skoro alocirani, imaju veliku šansu da budu uništeni.
- Mali objekti se češće uništavaju. Ako imamo program koji konstantno alocira i briše velike objekte, onda nešto nije u redu.

Kako radi ovaj sakupljač? Vršimo *Mark-and-sweep* i čuvamo brojač za svaki objekat koji označava koliko generacija su preživeli. Kada brojač dostigne određenu generaciju, vršimo prebacivanje. Opet čuvamo privremeni *forward* pokazivač do sledećeg *Mark*-a, kada se pokazivači ažuriraju. Najstariji objekti su oni koji su opstali najviše generacija. Ovaj *GC* ima osobinu da ređa slične tipove sa većom verovatnoćom jedan do drugog, što je korisno za keš.



Generacijski GC za programski jezik javu

Generalni problem sa *GC*-ovima je što svakako troše procesorsko vreme. Svi prethodni algoritmi zamrzavaju svet da bi mogli da počiste dubre. Uvek je jeftinije ne raditi ništa, ako nije potrebno da se rad. Bolje rešenje od *GC*-a ke da imamo sistem koji ne zahteva čišćenje preko njih. Jedna od alternativa je i sledeća metoda

VII Linearni tipovi

Ova metoda se zasniva na pretpostavci da vrednost **sme samo jednom** da se iskoristi. Nema potrebe za brojanjem referenci ili *GC*-a. Ovde nije dozvoljeno kopiranje, jer deterministički uništavamo tipove.

3. Parcijalna aplikacija i karijevanje

Funkcionalna paradigma se zasniva na transformacijama n -arnih funkcija f , u nove funkcije tako što funkciji f vežemo neke od argumenata za konkretne vrednosti. Ovo se naziva **parcijalna aplikacija**. **Karijevanje** je princip da je n -arna funkcija ona koja uzima *jedan element* i vraća $(n-1)$ - arnu funkciju, tj. **sve funkcije su unarne!**

Primer 1:

```
f x y z = x + y + z
f' = \x -> \y -> \z -> x + y + z
f' 1 2 3 = f1 2 3 , f1 y z = 1 + y + z
          = f2 3 , f2 z = 1 + 2 + z
          = f3 , f3 = 1 + 2 + 3 -- princip karijevanja
```

4. Algebarski tipovi podataka

Problem: Napraviti program koji za datu internet adresu vraća koliko reči ima na toj adresi.

U haskell-u:

```
url : String
brojac : Integer
web-page : Socket
-- Takodje su nam potrebne zastavice za stanje
started : Bool
finished : Bool
```

U C-u bi ovako enkapsuliralo u strukturu:

```
struct state_t {
    bool started = false;
    bool finished = false;
    unsigned count = 0;
    char* url;
    socket web_page;
};
```

Ovaj gornji pristup vuče ka mnogo bagova, mogućnosti propusta i dozvoljava nevalidna stanja.

Proizvod tipova A i B je tip čija vrednost sadrži tačno jednu vrednost tipa A i tačno jednu vrednost tipa B.

Primer 1(Spomenik miševi heroji): DNK je proizvod tipova koji gradi lanac. Moguće vrednosti su A,T,G i C. Broj mogućih lanaca DNK dužine n je 4^n . Veliki broj ovih lanaca je nevalidan. DNK predstavlja proizvod tipova. U najboljem slučaju ako nešto omašimo to biće neće biti rodjeno. U gorep slučaju možemo dobiti bagove. Ovaj primer je metafora za program i bagove u zavisnosti od mogućih stanja. Ako u programu napravimo tip koji ima mnogo veći broj nevalidnih stanja od validnih stanja, onda možemo da očekujemo probleme. Ovaj primer ilustruje problem koji smo naveli za url:

$$P = S \times B \times B \times I \text{ (Url} \times \text{Started} \times \text{Finished} \times \text{Brojač)}$$

Koje stanje za parove (*Started,Finished*) nije moguće? Odgovor je (*False,True*)

Zaključak je da su dovoljna tri stanja:

```
enum mode {
    initial,          // inicijalno stanje
    running,          // stanje brojanja
    finished           // krajnje stanje
};
```

Ako smo u inicijalnom stanju, nije nam potreban ni brojač ni soket, već samo *url* za povezivanje. Ako smo u stanju brojanja, onda nam je samo bitan brojač i soket, a u kranjem stanju je samo bitan brojač, tj. rezultat.

Treba voditi računa o curenju memorije prilikom prelaska iz stanja u stanje!

Sa ovim nadogradnjem, na žalost i dalje imamo nevalidna stanja. Umesto da pravimo proizvod tipova, možemo da imamo *sumu/uniju* tipova. Program je u našem slučaju ili *initial* ili *running* ili *finished*.

Suma tipova A i B je tip čija vrednost sadrži ili jednu vrednost tipa A ili jednu vrednost tipa B. U C++ imamo tip `std::variant` koja može da prima bilo niz vrednosti kojoj joj pridružimo.

Primer 2: Maybe

```
data Maybe a = Nothing | Just a
```

- `Nothing` je konstruktor koji nije parametrizovan, tj. nema argument. On ima tačno jednu vrednost.
- `Just a` je konstruktor koji je parametrizovan tipom `a`.
- Rezultat je jedna vrednost unija ceo skup `a`.

5. Teorija kategorija

Teorija kategorija je apstraktna teorija funkcija. Funkcije su usko vezane za skupove. Ako izbacimo skupove da definišemo nešto što liči na funkcije, tako dobijamo ovu nauku.

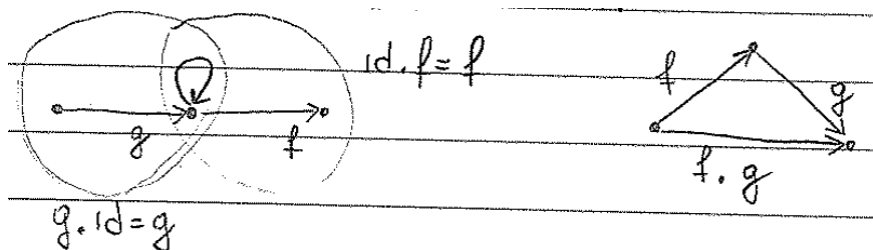
Definicija: Monograf je algebra koja se sastoji iz:

- *Objekata*: a, b, c
- *Strelica*: f, g, h - Strelice su isto objekti koji imaju *domen* i *kodomen*

$f: a \rightarrow b$ - f je strelica, a je domen, b je kodomen.

Definicija: Kategorija je monograf sa sledećim osobinama:

- *Identitet* - Ilti id dodeljuje strelicu samom sebi. Npr. $id \cdot f = f$, gde je $\cdot$ operacija kompozicije.
- *Kompozicija* - Dodeljuje strelicu $g \cdot f$ svakom paru strelica f i g sa $dom\ g = cod\ f$. Strelica $g \cdot f$ se naziva kompozicijom strelica g i f sa $g \cdot f: dom\ f \rightarrow cod\ g$. Kompozicija *mora* biti asocijativna.



Napomena: Svaka funkcija je strelica, svaka strelica nije funkcija.

Praktična kategorija u haskell-u:

- objekti su tipovi;
- strelice su funkcije;
- identitet je funkcija, `id`;

- kompozicija je operator .;

Definicija: Monoid (M, U, n) u kategoriji $\langle C, ., 1 \rangle$ je:

- množenje - $U : M.M \longrightarrow M$
- jedinica - $n : 1 \longrightarrow M$

U algebri je monoid struktura koja se sastoji iz nepraznog skupa i operacije koja je asocijativna. U monoidu imamo jedinstveni neutral, *mono id*.

U haskell-u je U - mappend, a n - mempty. Da vidimo pravila:

- $(\langle \rangle) = \text{mappend}$
- $x \langle \rangle \text{mempty} = x$
- $\text{mempty} \langle \rangle x = x$
- $(x \langle \rangle y) \langle \rangle z = x \langle \rangle (y \langle \rangle z)$

Primer 1: Niske

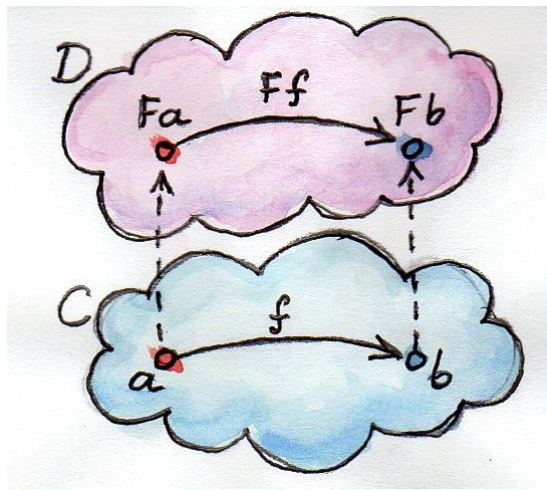
```
<String,++, "> -- moze i uopstenije
instance Monoid [a] where --za liste
    mappend = (++)
    mempty = []
```

Primer 2: Maybe

```
instance Monoid (Maybe a) where
    mappend Nothing x = x
    mappend x Nothing = x
    mappend (Just x1) (Just x2) = Just (mappend x1 x2)
    mempty = Nothing
```

Definicija: Funktor izmedju kategorija C i D je mapiranje objekata na objekte i strelice na strelice koje zadovoljavaju sledeća pravila:

1. $F(f) : F(\text{dom } f) \longrightarrow F(\text{cod } f)$ za svaku strelicu f u kategoriji C.
2. $F(g \cdot f) = F(g) \cdot F(f)$.
3. $F(1_a) = 1_{F(a)}$.



Na gornjoj slici vidimo pojavu koja se naziva *lifting*. Zamislimo da imamo kutiju u kojoj se nalazi broj n . Želimo da preslikamo broj n nekom unarnom funkcijom f u broj m ($f(n) = m$) **unutar kutije**. Da bismo to uradili potrebno je "otvoriti kutiju" i uzeti broj n , preslikati taj broj funkcijom f u broj m . Na kraju, broj koji smo dobili "zatvaramo u kutiju". Ceo ovaj *unwrap-and-wrap* proces se naziva lifting.

```
liftBox :: (a -> b) -> Box a -> Box b
```

Matematički:

$$\begin{aligned}
 f &: a \longrightarrow b \\
 F(f) &: F(a) \longrightarrow F(b) \\
 F(id_a) &= id_{F(a)} \\
 F(f \circ g) &= F(f) \circ F(g)
 \end{aligned}$$

U haskell-u je Functor sve što ima fmap:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
-- Maybe je u haskell-u funktor!
instance Functor Maybe where
    fmap func (Just val) = Just (func val)
    fmap func Nothing = Nothing
-- Primer primena monada
fmap (+3) (Just 2) == Just 5 <==> (+3) <$> (Just 2) == Just 5
```

Definicija: Endofunktor je funktor kategorije C na C . Primeri su liste, Maybe, Either.

Definicija: Monada M ($< M, return, join >$) nad kategorijom C se sastoji od endofunktora $M : C \longrightarrow C$ i dve transformacije $return : C \longrightarrow M(C)$ i operacije $join : M(M(C)) \longrightarrow M(C)$ koje zadovoljavaju sledeće:

- *levi identitet* - $return\ a \gg= f = fa$
- *desni identitet* - $m \gg= return = m$

- *asocijativnost* - $(m >>= f) >>= g == m >>= (\lambda x \rightarrow fx >>= g)$

gde je $>>=$ kompozicija `join` i `fmap`, koja se naziva *bind funkcija*.

```
class Monad m where
  return :: a -> m a
  (>>=) :: m a -> (a -> m b) -> m b
```

Šta ovo znači? Mi uzimamo neku funkciju, i primenjujemo je na vrednost koja se nalazi u kontekstu. Funkcija se primeni na vrednost i ona se ponovo vraća u taj kontekst.

```
-- Primer monada
half x = if even x
        then Just (x `div` 2)
        else Nothing
Just 3 >>= half == Nothing
Just 4 >>= half == Just 2
```

Fish operator ($>=>$) je operator koja pravi kompoziciji monadičkih vrednosti, ili kutija. Primenu funkcije i opet dobijamo zamotanu vrednost. Važi ista pravila kao i za monade.

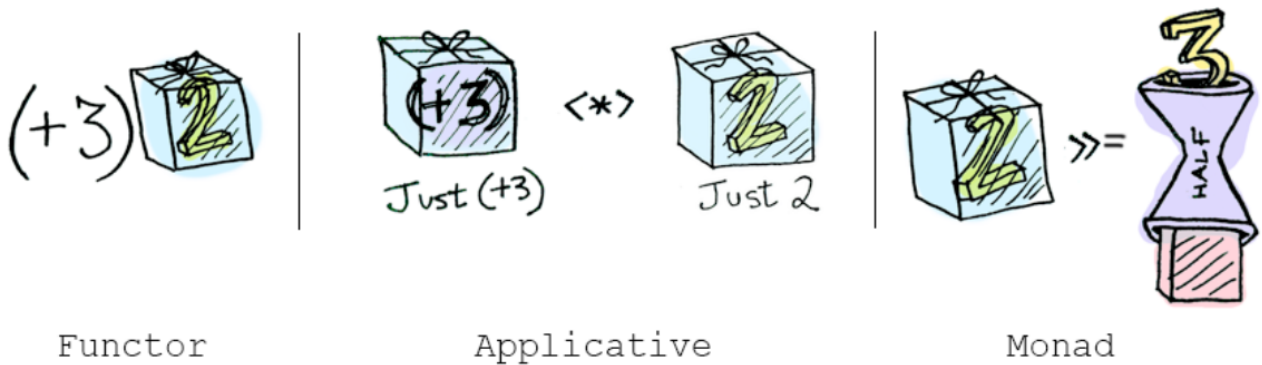
```
ispis : (>=>) :: (a -> mb) -> (b -> mc) -> a -> mc
```

Problem sa funktorima nastaje kada želimo da iskoristimo funkciju unutar kutije za mapiranje. Npr. imamo `Just (*3)` i `Just 5` i želimo da izvučemo `(*3)` iz prve funkcije i mapiramo `Just 5`. Ne možemo da mapiramo preko funkcije koja se nalazi u **funktoru neki funktor**. Taj problem rešavaju **aplikativni funktori**. Oni uzimaju funkciju iz prvog konteksta i primenjuju je na vrednost iz drugog konteksta, čime na kraju dobijamo vrednost primenjene funkcije umotana u kontekst.

```
class (function f) => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
```

Sada možemo da uradimo: `(Just (*3)) <$> (Just 5) = Just 15`, što nismo mogli da uradimo sa funktorima samo. `Pure` uzima vrednost i stavlja je u aplikativni funktor(kutiju). Aplikativni funktori predstavljaju nešto na šta možemo nastaviti ideju o monadama iako monade ne koriste to u svojoj definiciji.

```
-- Primer aplikativa
Just (+3) <*> Just 2 == Just 5
-- Posto je funktor, mozemo i ovo da uradimo
[(+2),(+3)] <*> [1, 2, 3] == [2,4,6,4,5,6]
```



Razlike izmedju funktora, aplikativa i monade

6. Zaključavanje tipova

6.1 U haskell-u

Obično definišemo tipove kad definišemo promenljive, npr:

```
x :: [Char]
i :: Int
x !! i :: Char;
```

Ovde nam pomažu **parametrizovani tipovi**, umesto da pišemo konkretni tip, imamo tipovske promenljive.

```
f :: a -> Bool
g :: a -> a -> a
```

U cpp-u je slično:

```
template <typename T>
T id(T val, std::vector<T> xs) { ... }
```

- prihvatamo jedan tip kao parametar tipa, iako prihvata dva argumenta

- ako znamo T, onda znamo oba argumenta funkcije
- slično kao i za funkciju u prethodnom primeru
- ovo se naziva *generičkim programiranjem*, polimorfizam preko tipova, *compile time* polimorfizam.

Kod jezika koji se kompajliraju, moramo dedukovati tipove tokom kompilacije. Tu nam dolaze u prilog **implicitni generički tipovi**.

Primer 1:

```
let f = \ x -> x
=> f :: a -> a -- jer dve iste promenljive => dva ista tipa
let g = f
=> g :: a -> a -- jer je f :: a -> a
let h = \ x -> f x -- neka je a = x, b = f x, c = f
-- iz prethodnog primera imamo da je f = \ x -> x
=> a = b
=> f :: a -> a , c = a -> a
=> h = a -> a
```

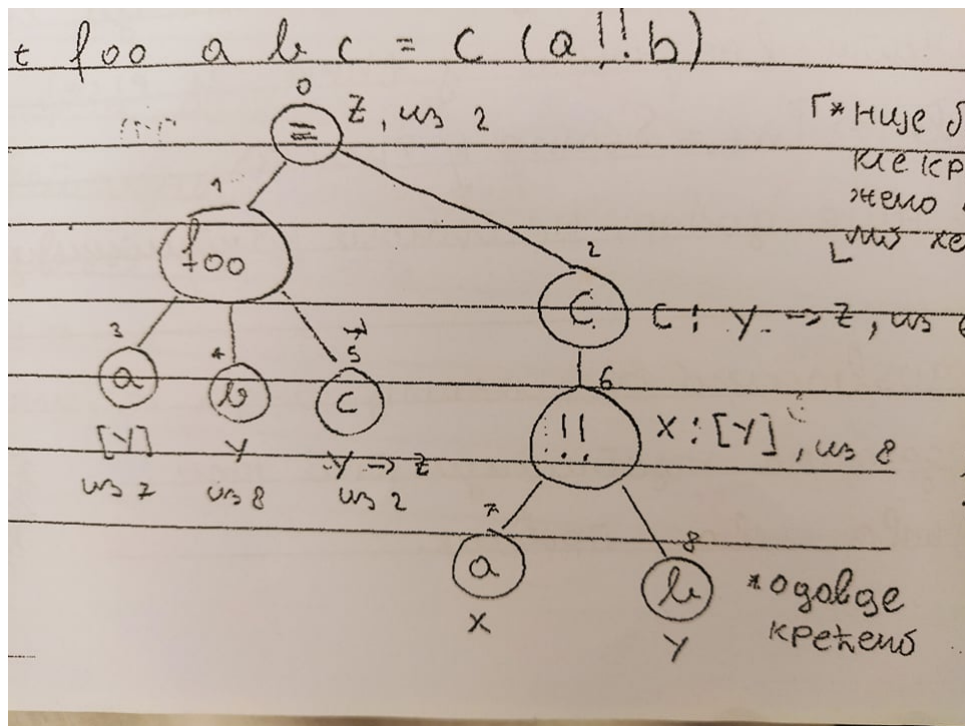
Primer 2: Pretpostavimo da nema klasa tipova

```
let max x y = if x > y then x else y
```

Šta se događa:

- Nema *overloading* => operator > može da radi samo za jedan tip x i jedan tip z. Primer (>) može da upoređuje *Int* i *String*.
- Neka *If* zahteva *Bool*
- Šta se dešava kada je (>) :: Int -> String -> Char? Dolazi do komplikovane greške, jer *If* ne dobija *Bool* koji zahteva.
- Ako je (>) :: Int -> String -> Bool onda se uslov ispunjava i ne dolazi do greške.

Primer 3: let foo a b c = c (a !! b)



$\Rightarrow \text{foo} :: [y] \rightarrow y \rightarrow (y \rightarrow z) \rightarrow z.$

Nije bitno odakle krećemo, možemo da se koristimo i heuristikama. Bitno je samo, ako dozvolimo overloading onda želimo da odredimo najopštiji tip koji ispunjava sva pravila.

Hindley-Milner provera tipova:

1. Zaključuje tipove koje nisu eksplicitno navedeni.
2. Jezik, konstruktori, ...
3. Unifikacija tipova.

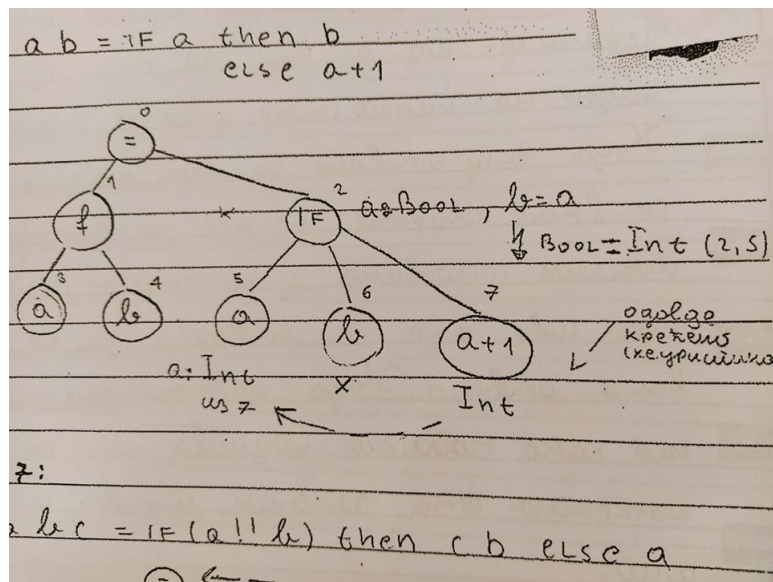
Neka zaključivanja:

- Za proizvoljnu binarnu operaciju nam je potrebno tri tipa ($a \rightarrow b \rightarrow c$)
- Ako je taj operator relacija, onda nam je potrebno dva tipa, jer je rezultat Bool ($a \rightarrow b \rightarrow \text{Bool}$)
- Operator $>$ poredi dva argumenta koji su istog tipa i vraća Bool ($a \rightarrow a \rightarrow \text{Bool}$)
- Operator $+$ koji sa leve strane a očekuje a, a sa desne strane i vraća a. Takođe možemo da očekujemo da je a neki numerički tip ($\text{Num} \rightarrow \text{Num} \rightarrow \text{Num}$).
- Operator $!!$ očekuje sa leve strane listu, a sa desne strane Int i kao rezultat, nešto što je istog tipa kao i elemenat liste.

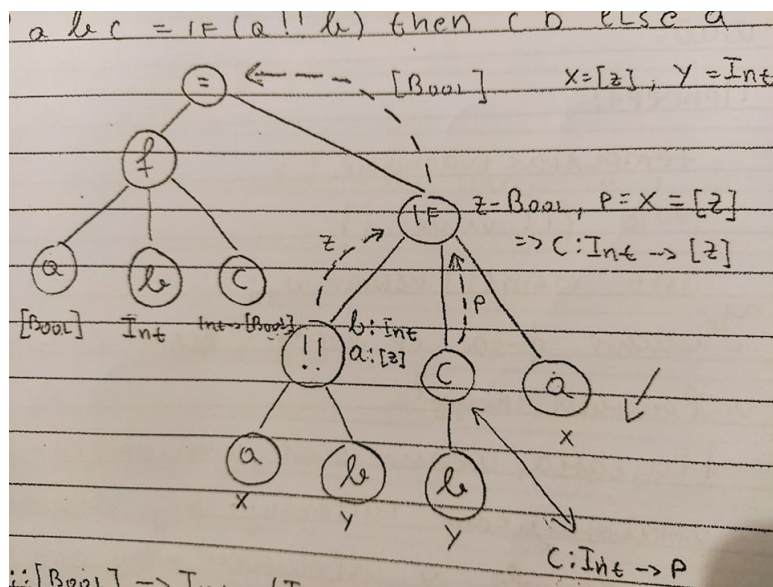
Kako radimo unifikaciju?

- Krećemo od korena drveta.
- Kada naletimo na konstruktor sa slobodnim tipovima, napravimo nove tipske parametre.
- Ako konstruktor ima dodeljena dva tipa T1 i T2, zameniti T1 i T2 u celom drvetu.

Primer 4: $\text{let } f \text{ a b} = \text{if } a \text{ then } b \text{ else } a + 1$



Primer 5: let f a b c = if (a !! b) then c b else a



6.2 U C++

U C++ kada definišemo *šablon*, on je definisan tako da radi za šta god da nama padne na pamet. Kada funkciju pozovemo za neki konkretan tip, tada se vrši kopiranje te funkcije sa zamenom parametarskim tipovima u konkretni tip. Kompajliranjem nove funkcije se proverava da li su ispoštovana pravila. Kada definišemo šablon, mi smo samo napisali uputstva kako se pravi neka konkretna funkcija zamenom tipova u konkretan tip. auto uglavnom radi slično kao šabloni.

Primer 1:

```
template<typename T>
void f(T x);
string s = "abc";
string &ss = s;
```

Dedukcija:

- $f(s) \sim T = \text{string}$
- $f(ss) \sim T = \text{string}$

Uglavnom šablon uzima isti tip za T i sklanja reference, const...

Primer 2:

```
template<typename T>
void f(T&& value) {
    g(std::forward<T>(value));
}
```

- Ovo može da se poziva sa običnom vrednošću (sa referencom) ili sa privremenom && vrednošću.
- Potrebno je još da u funkciju *g* prosledimo isto što smo dobili. Zato koristimo *forwarding reference* što radi baš to što želimo. Prosleđivanje referenci je posebna vrsta referenci koje čuvaju vrednosnu kategoriju argumenta funkcije, što omogućuje njegovo prosljeđivanje.